CSCI 164

Final Project Report

Applied Machine Learning with scikit-learn

By: Laura Mas Serra, Kazi Islam, and Manuel Domingo Mora

1. Introduction

Machine learning has become a widely used approach for solving real-world predictive problems by learning patterns from data. In this project, we apply classical supervised learning techniques using the scikit-learn library to build, evaluate, and compare multiple classification models across different datasets. The main objective is to demonstrate a complete machine learning workflow, from data preprocessing to model evaluation, and to analyze how different algorithms perform under varying data conditions.

Each dataset was carefully explored to understand its structure, including feature types, target variables, and overall data distribution. Standard preprocessing techniques were applied, such as handling missing values, encoding categorical variables into numerical form, and feature scaling where necessary. In particular, the bank marketing dataset and the weather forecast dataset present significant class imbalance problems, where one class is heavily underrepresented. This provides an opportunity to analyze how imbalance affects model performance and how techniques such as resampling and class weighting can improve minority class prediction.

We implemented and compared three supervised learning algorithms: Logistic Regression, Decision Tree, and K-Nearest Neighbors (KNN). These models were selected due to their interpretability, efficiency, and strong baseline performance on tabular datasets. Each model was trained and evaluated using train-test splits, and hyperparameter tuning was performed using GridSearchCV to optimize performance.

Moreover, model evaluation was conducted using multiple metrics, including accuracy, precision, recall, F1-score, ROC-AUC, and confusion matrices. These metrics provide a more comprehensive understanding of performance beyond simple accuracy, especially for the

aforementioned imbalanced datasets. Visualizations were also used to support interpretation of results and highlight differences between models.

Lastly, we performed a literature review of earlier studies employing the bank telemarketing dataset to get a sense of benchmarks, especially the work by Sérgio Moro, Paulo Cortez, and Paulo Rita., which uses more complex models like Neural Networks and Support Vector Machines. In particular, they measured success with AUC and LIFT and used a rolling window validation. By lining up our results with theirs, we got a clearer picture of how things like model complexity, picking certain features, or choosing an evaluation method can really shift outcomes. It also gave us a way to benchmark how our own results land compared to other implementations on the same dataset.

2. Dataset Exploration

Phishing Dataset Overview and Data Preprocessing :

This dataset was selected because it addresses a significant real-world cybersecurity problem: phishing attacks. These remain one of the most common ways of cyber fraud and are a risk to individuals and organizations. The dataset is well-structured and commonly used in machine learning research, making it suitable for classification tasks, while ML continues to play a major role in cybersecurity.

This phishing dataset contains information about websites and is designed to classify whether a website is legitimate or phishing. Each row represents an individual website instance, while each column corresponds to a specific feature related to website characteristics, including URL properties, security indicators, and domain-related attributes

The target variable is called “class”, where:

- 1 = legitimate website
- -1 = phishing website

The data set is not fully ready to use yet so before training the models, several preprocessing steps were applied in order to successfully train the model. First, we converted the target variable

from -1 and 1 to 0 and 1. This is a very important part because most machine learning models work better with labels in the form of 0 and 1. Second, the dataset was checked for missing values. Any missing numeric values were replaced using the median value of each column. This helps to avoid errors during model training. Third, the dataset features were separated into two different parts such as input features x and the target variable y. Then the data was split into training and testing sets to evaluate model performance. Finally, StandardScaler used to apply normalization. This step scales the features so that they are on a similar range and the data points are not far from each other. Normalization is especially important for models such as Logistic Regression and KNN.

Since all features in this dataset are already numeric, no encoding was required. The dataset does not contain significant missing values, so only minimal cleaning was required. Basic statistical analysis and dataset information were explored using functions such as head(), info(), and describe() to understand the data structure.

Overall, the phishing dataset required basic preprocessing steps such as label conversion, handling missing values, and normalization before applying machine learning models.The phishing dataset is clean and structured, but the features are already engineered, which limits deeper feature analysis.

Bank Marketing Dataset Overview and Data Preprocessing :

This dataset was selected because marketing plays a key role in Management Information Systems (MIS) as it allows analyzing customer behavior and supporting business decision-making. Predicting whether a client will subscribe to a term deposit can help financial institutions improve marketing strategies. The dataset is also valuable from a machine learning perspective because it includes class imbalance, allowing us to study model performance under challenging conditions.

The bank marketing dataset contains information about bank customers and is commonly used for classification tasks. It includes 41,188 records and 21 features, with the objective of predicting whether a client will subscribe to a term deposit. Each row represents an individual customer, while each column corresponds to a specific attribute related to that customer. The dataset includes various types of information:

- Customer information such as age, job, marital status, education
- Financial information such as loan, housing, default
- Contact details such as contact type, month, day of week, duration of call
- Economic indicators such as employment rate, consumer price index, euribor rate

The target variable in this dataset is “y”, which shows whether the customer subscribed to the term deposit or not. The values are:

- yes = client subscribed
- no = client did not subscribe

This dataset has both numerical and categorical data this makes it more complex compared to the phishing dataset. The categorical features include job, marital status, and education, while the numerical features include age, duration, and economic indicators. Because of this mixed data structure, the models must effectively process both feature types in order to generate accurate predictions.

Several preprocessing steps were required before model training. First, the dataset was imported using pd.read_csv() with sep=";" because the original file used semicolons rather than commas as separators. Second, we checked the dataset for missing values, and any missing numerical values were handled using `fillna()` to prevent issues during training. Third, the target variable “y” was converted from text values “yes” and “no” into binary numeric values 1 and 0 using the map() function. Fourth, the features were separated into input features X and target variable y using the drop() function. Fifth, encoding was necessary to take care of the categorical data, using the function pd.get_dummies(), so that ML models can properly interpret them. Sixth, we used the function train_test_split() to split into training and testing sets to prepare it for model development and evaluation. Finally, the function StandardScaler() with fit_transform() and transform() was used to normalize data. This helps us to keep all features in a similar range. This is especially important for Logistic Regression and KNN models. Overall, all of these processing steps were required to get accurate results and have better model performance.

Weather Forecast Overview and Data Preprocessing :

This dataset was selected because it represents a practical prediction problem related to weather forecasting. Predicting rainfall is important for decision making in agriculture and transportation. The dataset is also simple and well-structured, making it useful for testing classification models. The next dataset selected was the Weather Forecast Dataset obtained from Kaggle. This dataset was chosen because it presents a practical binary classification problem where the goal is to predict whether rainfall will occur based on meteorological conditions. This topic becomes relevant because accurate weather forecasting can support decision making in agriculture, transportation, emergency planning, and environmental monitoring.

The dataset contains a total of 2,500 instances and six variables: five numerical input features including temperature, humidity, wind speed, cloud cover, and atmospheric pressure; and one categorical target variable named “Rain” indicating whether an instance resulted in rainfall or not.

To better understand the structure of the dataset, we used several exploratory functions gaining insight into the dimensionality, features and data types, and summary statistics. We also assessed the quality of the data by identifying missing values and duplicate instances, which determined that none of those existed. Despite the dataset not requiring major cleaning, preprocessing procedures for handling missing values and duplicates were still included for completeness: numerical variables were configured to use median imputation in the case of missing values, and categorical variables were assigned mode imputation. Including these steps ensures that the workflow is adaptable to future work.

Additionally, several plots were displayed to further describe the nature of the dataset features:. A pairplot was generated to examine pairwise relationships between numerical features based on rainfall predictions, which suggested that variables such as humidity and cloud cover may have stronger relationships with rainfall compared to other features. While these patterns were not perfectly separable, they indicate the existence of meaningful predictive relationships. Moreover, a correlation heatmap showed that moderate correlations existed between features, but no extremely high correlations, which allows us to determine that multicollinearity is not a major concern within this dataset, allowing all features to be retained for model training. Lastly, we displayed histograms to examine feature distributions, which showed to be balanced without

severe skewness or extreme outliers, suggesting that the numerical variables are suitable for machine learning algorithms.

One important consideration is the detection of class imbalance within the target variable, with a countplot showing a noticeably larger number of “no rain” compared to “rain” instances. To address this issue and avoid bias, we used stratified sampling during train-test splitting to preserve the original class proportions. In addition, a complete metric evaluation was used to provide a comprehensive assessment of model performance. On another note, the target variable was converted into numerical form using LabelEncoder, transforming rainfall categories into binary labels (rain = 1, no rain = 0), and we standardized the numerical features using StandardScaler. This step was particularly important for algorithms such as Logistic Regression and K-Nearest Neighbors, which are sensitive to differences in feature magnitude. Finally, the dataset was split into training and testing sets using an 80/20 ratio, with a fixed random state of 42 to ensure reproducibility.

3. Model Development with scikit-learn

This project employs three supervised machine learning algorithms for classification tasks: Logistic Regression, Decision Tree, and K-Nearest Neighbors (KNN). These models were selected in accordance with the project requirements and due to their strong suitability for classification problems, as well as their reliable availability on Kaggle and ease of implementation in the scikit-learn library.

Before training any model, the dataset was prepared using standard preprocessing steps. Missing values and duplications were handled where necessary, categorical variables were encoded into numerical form, and features were scaled using StandardScaler() to ensure that all variables were on a comparable range. The dataset was then split into training and testing sets using train_test_split(), which allows the models to be trained on one portion of the data and evaluated on unseen data for a fair assessment of performance.

First, Logistic Regression is a linear classification algorithm widely used for binary and multi-class classification problems, modelling the probability of a class label. Despite its

simplicity, Logistic Regression is effective when the relationship between features and the target variable is approximately linear and serves as a strong baseline model. Logistic Regression was implemented by using the function LogisticRegression(max_iter=1000). We set the max iteration to 1000. This parameter sets the maximum number of iterations for the optimization process to 1000, ensuring that the algorithm has sufficient steps to converge, especially for datasets with complex feature interactions. Lastly, the function fit() is used to obtain a model and the function predict() on the test data.

Second, the Decision Tree model is a non-linear, rule-based algorithm that recursively splits the dataset based on feature values to form a tree-like structure. Each internal node represents a decision rule, while each leaf node represents a predicted class label. Decision Trees are highly interpretable and can capture complex, non-linear relationships in the data, but are prone to overfitting the data if not properly tuned. We implemented Decision Tree by using the function DecisionTreeClassifier(random_state=42). The random_state parameter was set to ensure reproducibility of results by controlling randomness during the splitting process. As mentioned earlier, it creates a tree based model that splits the data into smaller groups based on feature values using decision rules. The model learns by creating branches, where each branch represents a condition for example, age > 30, and continues splitting until it reaches a final decision. Decision Trees were chosen due to their interpretability and ability to handle both numerical and categorical features without requiring strong assumptions about the underlying data distribution, and their structure allows the model to capture complex, non-linear relationships in the data.

Third, K-Nearest Neighbors (KNN) is an instance-based, non-parametric learning algorithm that classifies a data point based on the majority class among its k closest neighbors in the feature space. Distance metrics such as Euclidean distance are typically used to determine similarity between data points. KNN is simple and intuitive but can be computationally expensive during prediction and sensitive to feature scaling and the choice of k. KNN was implemented by initializing it using KNeighborsClassifier(). As mentioned earlier, prior to applying the model, feature scaling was performed using, especially important in the case of KNN, to ensure that all variables contribute equally to distance calculations, as it is a highly sensitive model to differences in feature magnitude. This model was chosen due to its simplicity and ability to

model complex, non-linear decision boundaries without requiring assumptions about the underlying data distribution. However, it can be computationally expensive during prediction for large datasets and, again, is sensitive to irrelevant or improperly scaled features, which highlights the importance of preprocessing for effective performance.

4. Hyperparameter Tuning

To improve model performance and ensure fairness in the comparison across classifiers, we performed hyperparameter tuning using GridSearchCV with 5-fold cross-validation (). This method evaluates multiple combinations of hyperparameters for each model and selects the configuration that achieves, in our case, the best performance based on the ROC-AUC score. Using cross-validation reduces the risk of overfitting and provides a more reliable model generalization.

Each model was tuned by defining a specific parameter grid based on its structure and behavior. Logistic regression was tuned by adjusting regularization strength and solver type, resulting in C = 1 and solver = liblinear, which applies a moderate level of regularization controlling model complexity and improves generalization to unseen data. For the Decision Tree model, the best parameters were max_depth = 3 and min_samples_split = 2. The maximum depth being 3 reduces the complexity of the tree and helps prevent overfitting, resulting in a more interpretable model. Lastly, For the K-Nearest Neighbors (KNN) model, the optimal parameters were n_neighbors = 13 and weights = distance, where a large k improves the stability of predictions. Also, using distance-based weighting ensures that closer neighbors have a stronger influence on the final classification, which improves performance in regions where class boundaries might be denser or more complex.

5. Results and Evaluation

Metrics Used:

To successfully evaluate and compare the performance of the classification models, we implemented standard metrics, providing insights into model behavior, especially in the case of class distribution imbalances. Accuracy measures the proportion of correctly classified instances out of all predictions, and while it provides an overall indication of performance, it can be misleading when classes are imbalanced. Precision measures the proportion of correctly predicted positive instances out of all instances predicted as positive and is particularly important when false positives are costly. In our case, we use weighted precision to address the class imbalance. Recall measures the proportion of actual positive instances that were correctly identified by the model, and becomes really relevant when missing positives is not desired. F1-score is the harmonic mean of precision and recall. It provides a balanced measure of model performance when there is a trade-off between precision and recall, and is particularly useful for imbalanced datasets like some of ours. Lastly, ROC-AUC evaluates the model's ability to distinguish between classes across different classification thresholds. It is based on predicted probabilities rather than discrete class labels and provides a threshold-independent measure of separability, where a higher ROC-AUC indicates better discrimination between positive and negative classes.

Dataset Phishing Website Detection Results:

This section presents the performance results of the machine learning models based on the conducted experiments. As mentioned earlier, the report includes three models: Logistic Regression, Decision Tree, and K-Nearest Neighbors (KNN); and to provide a comprehensive comparison we employ the metrics including accuracy, precision, recall, and F1-score

```
=== Model Comparison (Accuracy) ===
Logistic Regression: 0.9326096788783356
Decision Tree: 0.952962460425147
KNN: 0.9466304839439168
```

Table 1: Phishing Dataset Accuracy Results.

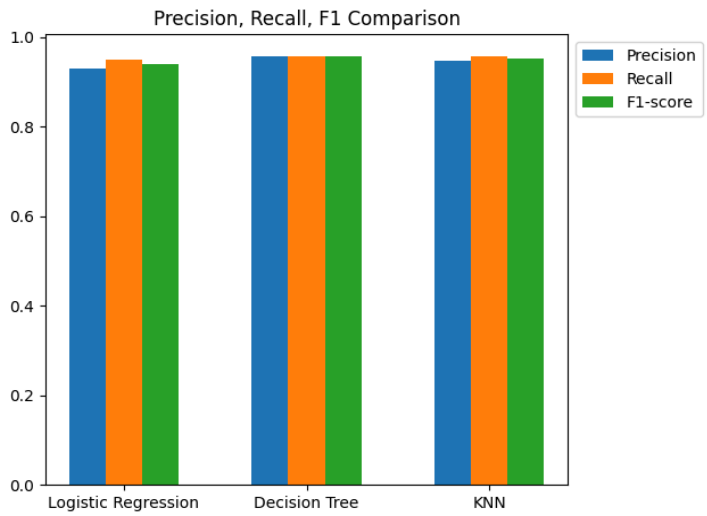

Figure 1: Phishing Dataset Metric Results.

Based on the results, the Decision tree model achieved the highest overall accuracy of 95.3%. It also demonstrated superior performance scores across the remaining metrics. KNN follows closely with an accuracy of 94.66% and consistent reliability across all metrics, while Logistic Regressions results on a strong baseline despite having the lowest accuracy of 93.26%.

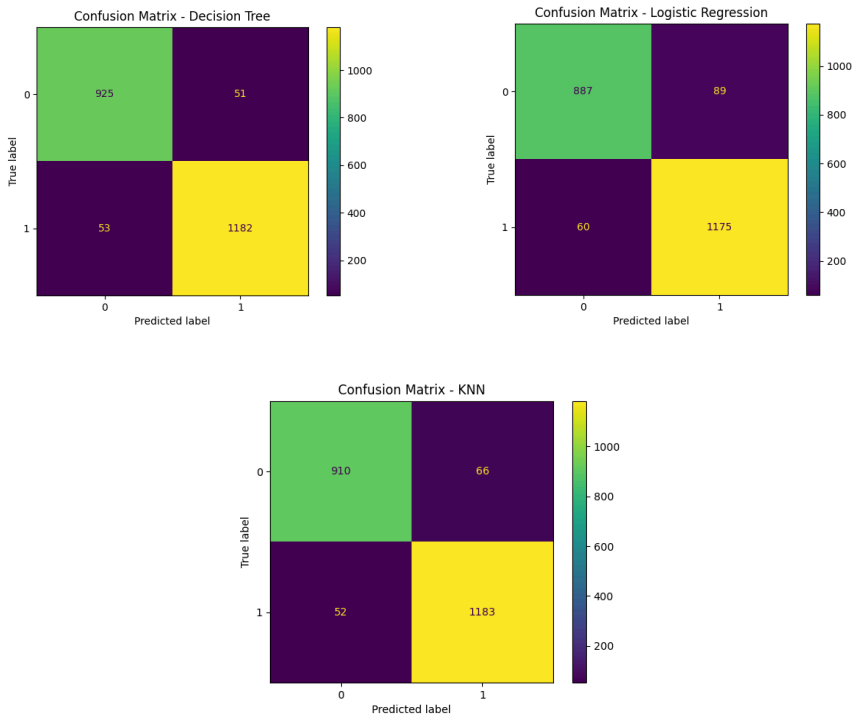

Figure 2: Confusion Matrices for all 3 models (Phishing Dataset).

The confusion matrix analysis further solidifies the Decision Tree as the most precise model, as it minimizes total misclassifications with only 104 errors (51 False Positives and 53 False Negatives). In contrast, Logistic Regression exhibited the highest degree of error, particularly struggling with Type I errors by incorrectly labeling 89 instances of Class 0 as Class 1. While KNN achieved the highest number of True Positives (1183), it suffered from a higher count of False Positives (66) compared to the Decision Tree. Moreover, the "True Positive" diagonal (bottom-right) consistently shows higher values than the "True Negative" diagonal (top-left) across all three models, indicating a dataset with a slight class imbalance that all models successfully navigated and keeping strong predictive performance for the majority class.

The ROC curve shows the relationship between the True Positive Rate and the False Positive Rate for different threshold values. All three models perform very well because their curves are close to the top-left corner.. Logistic Regression achieved an AUC of 0.98, KNN achieved 0.981, and Decision Tree achieved 0.973. These values are very close to 1,. Among the three models, KNN has the highest AUC value, followed closely by Logistic Regression, while Decision Tree has a slightly lower AUC. all models have excellent classification performance.Overall, the ROC curve confirms that the phishing dataset is well-structured and easy to classify, and all three models are able to achieve high prediction accuracy.

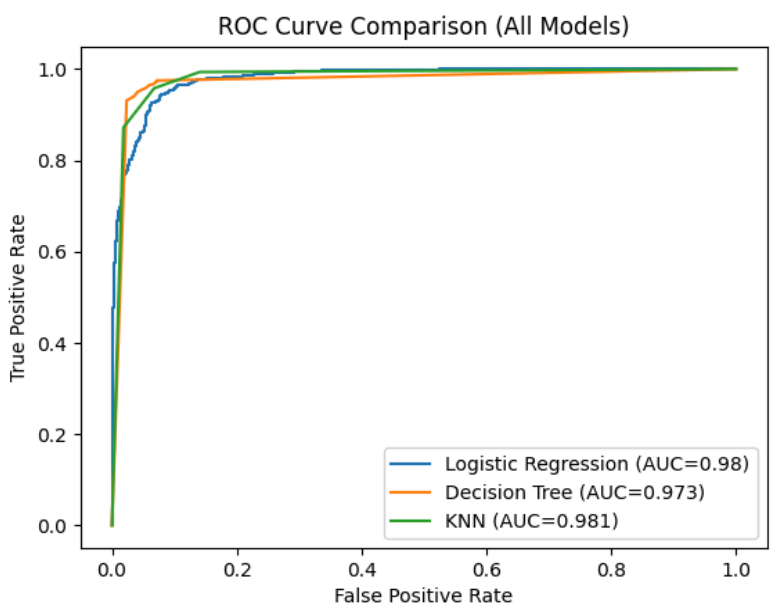

Figure 3: ROC Curves (Phishing Dataset).

In this project, hyperparameter tuning was applied to improve the performance of all three models. The tuning process was done using the GridSearchCV() function, which tests different parameter combinations and selects the best model based on performance. Logistic Regression, parameters such as C as regularization strength and solver were tuned. For the Decision Tree, parameters such as max_depth and min_samples_split were adjusted. For KNN, the number of neighbors" n_neighbors" was tuned.

For Logistic Regression, the performance remained almost the same. Accuracy slightly increased from 0.9322 to 0.9326, and small improvements were observed in precision and F1-score. Decision Tree, the performance slightly decreased after tuning. Accuracy dropped from 0.9579 to 0.9539, and small decreases were observed in precision, recall, and F1-score. KNN tuning showed the most improvement. Accuracy increased from 0.9389 to 0.9475, and improvements were observed in precision, recall, F1-score, and AUC.
Logistic regression shows that the default model was already well-optimized, and tuning only made minor improvements. The original parameters of the decision tree were already performing very well, and tuning did not improve the model further.KNN is more sensitive to parameter changes, and tuning helped improve its performance.

The results also show that different models respond differently to tuning. KNN benefited the most from tuning, while Logistic Regression and Decision Tree showed only small changes.hyperparameter tuning improved the performance of some models and had minimal or negative impact on others.

```
=== BEFORE vs AFTER TUNING COMPARISON ===

Model                Stage    Accuracy   Precision  Recall     F1-score   AUC
--------------------------------------------------------------------------------
Logistic Regression  Before   0.9322     0.9282     0.9522     0.94       0.9802
Logistic Regression  After    0.9326     0.9296     0.9514     0.9404     0.98
Decision Tree        Before   0.9579     0.9612     0.9636     0.9624     0.9572
Decision Tree        After    0.9539     0.9558     0.9619     0.9588     0.9547
KNN                  Before   0.9389     0.9386     0.953      0.9458     0.9825
KNN                  After    0.9475     0.9501     0.9563     0.9532     0.9876
```

Table 2: Before and after Hyperparameter Tuning results.

Dataset Telemarketing Results:

To successfully evaluate and compare the performance of the classification models, we implemented standard metrics such as accuracy, precision, recall, and F1-score. These metrics provide insights into model behavior, especially in the case of class imbalance, which is present in this dataset.

In this section, we report the performance of three models: Logistic Regression, Decision Tree, and KNN, based on the experimental results.

Based on the comparison, the Decision Tree model achieved the highest accuracy of 0.915, followed by Logistic Regression with 0.911, and KNN with 0.897. This shows that Decision Tree performs slightly better than the other models in terms of overall accuracy.

```
=== Model Comparison (Accuracy) ===
Logistic Regression: 0.9112648701141054
Decision Tree: 0.9150279193979121
KNN: 0.8976693372177713
```

Table 3: Telemarketing Dataset Accuracy Results.

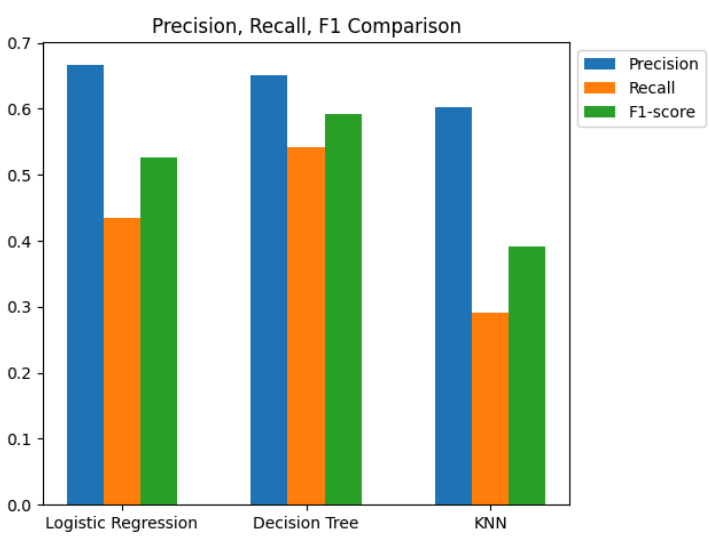

Figure 4: Telemarketing Dataset Metric Results.

Looking at the classification report for Logistic Regression, the model performs very well for class 0, with precision and recall values around 0.93 to 0.97. However, for class 1 , the recall is low at 0.44, which means the model fails to identify many customers who actually subscribed. Precision for class 1 is 0.67, which shows that when the model predicts “1” it is often correct, but it misses many true positive cases. It has TN= 7100, FP= 203, FN = 528, TP = 407. The model is very good at predicting 0 but it misses many 1s.

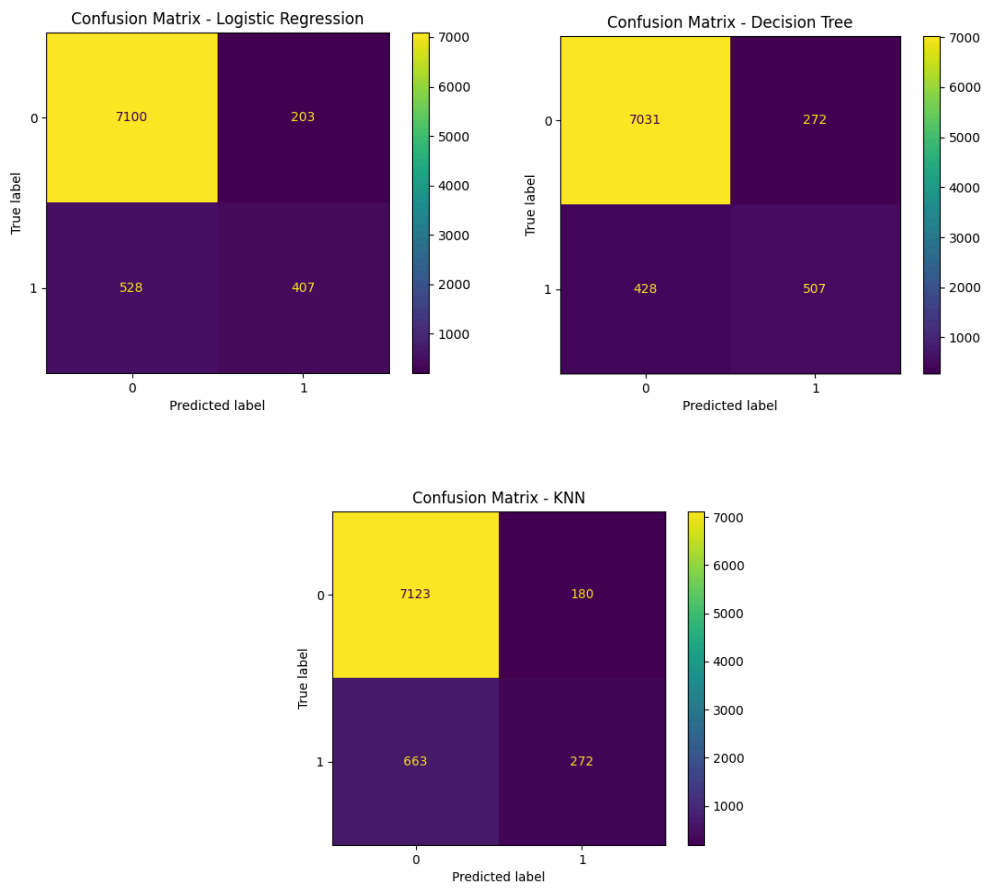

Figure 5: Confusion Matrices for all 3 models (Telemarketing Dataset).

For the Decision Tree model, the performance is more balanced. Precision for class 1 is 0.65, and recall is 0.54, with an F1-score of 0.59. This shows that Decision Tree performs better than Logistic Regression in identifying customers who subscribed. It also maintains high performance for class 0 with precision and recall values around 0.94 to 0.96. Overall, Decision Tree provides a better balance between precision and recall. It has TP=407, FN=528, FP=203, TN= 528 this

means avoiding false alarms, but misses real cases. It has TP= 507, FN = 428, FP = 272, TN= 7031 indicating more real positives but gives more false alarms.The Decision Tree model provides a better balance between precision and recall compared to the other models. It reduces the number of missed positive cases while still maintaining strong performance for the majority class. This makes it more suitable for this dataset, especially because identifying customers who are likely to subscribe is more important than avoiding a small number of false positives.

The KNN model shows the lowest performance among the three models. While it performs very well for class 0 with recall as high as 0.98, its performance for class 1 is weak. Recall for class 1 is only 0.29, which means the model misses most of the positive cases. Precision is also lower at 0.60, and the F1-score is 0.39, which indicates poor performance for the minority class.It has TP = 272 , FN = 663 ,FP = 180 , TN = 7123 which indicate the model misses too many real positives.

One important observation is that this dataset is imbalanced, with many more 0 responses than 1 responses. Because of this, accuracy alone is not a reliable measure. Models can achieve high accuracy by predicting the majority class, but fail to detect the minority class. Therefore, recall and F1-score for class 1 are more important in this case. Overall, the Decision Tree model provides the best performance for this dataset because it achieves the highest accuracy and shows a better balance between precision and recall for the important class 1. Logistic Regression performs reasonably well but struggles with recall, while KNN performs poorly for the minority class.

In this project, hyperparameter tuning was applied to improve the performance of all three models. The tuning process was done using the GridSearchCV() function, which tests different parameter combinations and selects the best model based on performance. For Logistic Regression, parameters such as C which is regularization strength and solver were tuned. For the Decision Tree, parameters such as max_depth and min_samples_split were adjusted. For KNN, the number of neighbors n_neighbors was tuned. For Logistic Regression, the performance remained almost the same after tuning. Accuracy slightly decreased from 0.9115 to 0.9113, and small changes were observed in precision and recall. The F1-score remained the same at 0.5269,

and AUC also stayed almost unchanged. This shows that the model was already well-optimized, and tuning did not significantly improve its performance.

```
=== BEFORE vs AFTER TUNING COMPARISON ===

Model                Stage   Accuracy  Precision  Recall  F1-score  AUC
---------------------------------------------------------------------
Logistic Regression  Before  0.9115    0.67       0.4342  0.5269    0.9349
Logistic Regression  After   0.9113    0.6672     0.4353  0.5269    0.9348
Decision Tree        Before  0.8863    0.499      0.5209  0.5097    0.7269
Decision Tree        After   0.915     0.6508     0.5422  0.5916    0.9212
KNN                  Before  0.8972    0.5791     0.3444  0.4319    0.8145
KNN                  After   0.8977    0.6018     0.2909  0.3922    0.8448
```

Table 4: Before and after Hyperparameter Tuning results.

For the Decision Tree, tuning showed a strong improvement. Accuracy increased from 0.8863 to 0.915, and precision improved from 0.499 to 0.6508. Recall also increased slightly from 0.5209 to 0.5422, and the F1-score improved from 0.5097 to 0.5916. AUC also increased significantly from 0.7269 to 0.9212. This shows that Decision Tree benefited a lot from tuning, and the new parameters helped the model perform much better.

For the KNN model, tuning showed mixed results. Accuracy increased slightly from 0.8972 to 0.8977, and precision improved from 0.5791 to 0.6018. However, recall decreased from 0.3444 to 0.2909, and the F1-score dropped from 0.4319 to 0.3922. AUC improved slightly from 0.8145 to 0.8448. This shows that tuning improved some aspects of the model but reduced its ability to detect positive cases.

Overall, hyperparameter tuning had different effects on each model. Decision Tree benefited the most from tuning and showed significant improvement in all metrics. Logistic Regression showed almost no change, indicating it was already well-optimized. KNN showed small improvements in accuracy and precision but performed worse in recall and F1-score.

The results show that different models respond differently to tuning. Decision Tree benefited the most, while Logistic Regression showed minimal changes and KNN showed mixed performance.

Hyperparameter tuning improved the performance of some models and had minimal or negative impact on others.

Dataset Weather Forecast Results:

The following Table 5 summarizes the performance of each model before and after hyperparameter tuning. For Logistic Regression and Decision Tree, metrics remained the same, indicating that the scikit-learn settings had already captured the optimal decision boundaries. K-Nearest Neighbors (KNN) was the only model that showed improvement in ROC-AUC after hyperparameter tuning, so the number of neighbors and the weight functions offered a better probability separation between the classes.

BEFORE TUNING

	accuracy	precision	recall	f1	roc_auc
Logistic Regression	0.926	0.924573	0.926	0.925224	0.964912
Decision Tree	0.998	0.998031	0.998	0.998007	0.998856
KNN	0.964	0.963567	0.964	0.963751	0.982020

AFTER TUNING

	accuracy	precision	recall	f1	roc_auc
Logistic Regression	0.926	0.924573	0.926	0.925224	0.964912
Decision Tree	0.998	0.998031	0.998	0.998007	0.998856
KNN	0.964	0.964000	0.964	0.964000	0.993426

Table 5: Model Performance Before and After Tuning

Ultimately, this analysis suggests that the initial parameters were already well-suited and suggesting that this dataset is very easily separable, and might have undergone heavy data cleaning and simplification before publication, or even a synthetic nature.

	accuracy	precision	recall	f1	roc_auc	combined_score	rank
Decision Tree	0.998	0.998031	0.998	0.998007	0.998856	0.998431	1.0
KNN	0.964	0.964000	0.964	0.964000	0.993426	0.978713	2.0
Logistic Regression	0.926	0.924573	0.926	0.925224	0.964912	0.945068	3.0

Table 6: Final Model Ranking

More importantly, Table 6 provides a Final Model Ranking, based on a combined evaluation of F1-score and ROC-AUC, identifying the Decision Tree as the superior classifier for this dataset. Its first-place is a direct result of its near-perfect accuracy and ability to maximize the area under the ROC curve. KNN follows in second place, demonstrating highly stable and competitive performance that benefited slightly from hyperparameter optimization. Logistic Regression ranks third; while it remains a viable and computationally efficient baseline, its linear nature prevents it from matching the precision of the other two models.

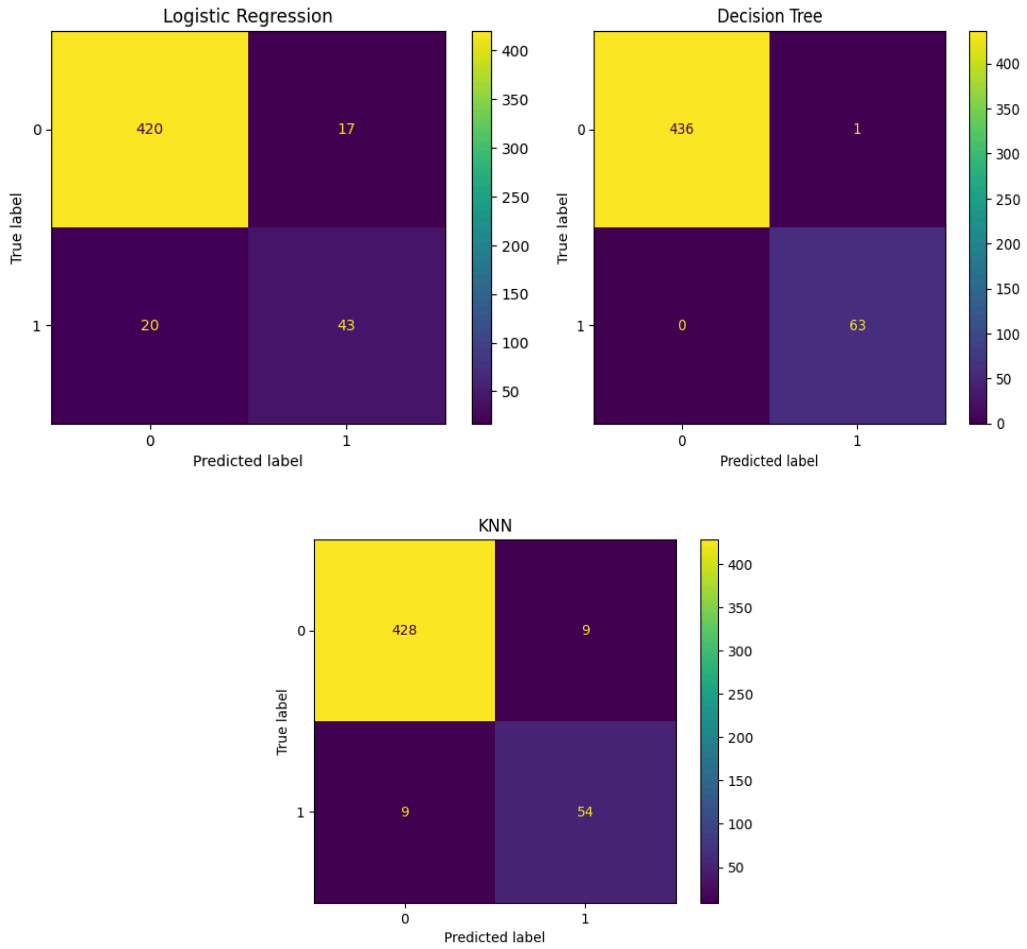

Figure 6: Confusion Matrices for all 3 models (Weather Forecast Dataset)

The confusion matrices reported in Figure 6 showed that Decision Tree achieved near-perfection performance, correctly identifying 499 out of 500 test samples, producing only one error (a False Positive); KNN showed balance performance with 18 total errors, which interestingly, displayed symmetry in its mistakes (with 9 False Positives and 9 False Negatives); Lastly, Logistic

Regression struggled the most with the minority class “Rain”, failing to identify 20 out of the 63 actual rain instances, indicating that a linear boundary is less effective for this weather data. This trend suggests that the Decision Tree is far more adept at handling the class imbalance present in this specific dataset, whereas the other models lean toward a bias for the majority class (Class 0), resulting in lower recall for the target class.

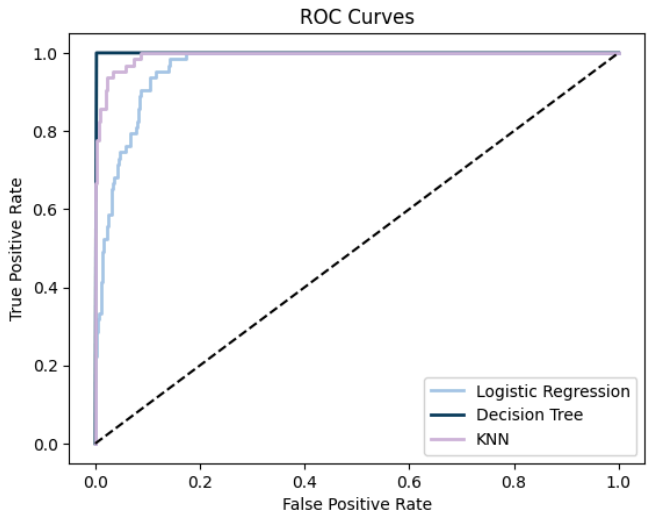

Figure 7: Predictive Separation (ROC Curves) of all 3 models.

The ROC analysis shown in the graph below (Figure 7) confirms the Decision Tree as the top performing model with an AUC of almost 1.0. Then KNN followed closely, but Logistic Regression required a higher trade-off in False Positives to achieve a similarly high True Positive rate, ranking its curve the lowest of the three.

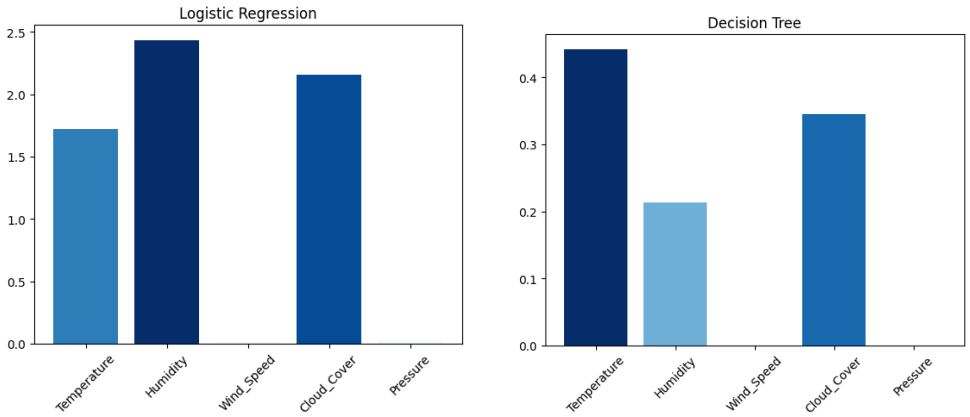

Figure 8: Feature Influence

The feature importance plots (Figure 8) reveal a clear disagreement between our models, which might explain the performance gap seen earlier: Decision Tree heavily prioritizes Temperature as its high accuracy suggests that this feature likely has a "threshold" effect (a specific cutoff point) that is excellent for separating the classes. Logistic Regression, on the other hand, relies mostly on Humidity and Cloud Cover, and as it looks for a straight-line relationship, it misses the complex Temperature patterns that the Tree model used to achieve higher accuracy.

It is important to note that K-Nearest Neighbors does not provide intrinsic feature importance because it is a non-parametric, instance-based learning algorithm. Unlike tree-based models or linear models, it does not learn explicit feature weights or coefficients. Therefore, feature importance analysis is not directly applicable and is omitted for this model to maintain consistency in interpretation across models.

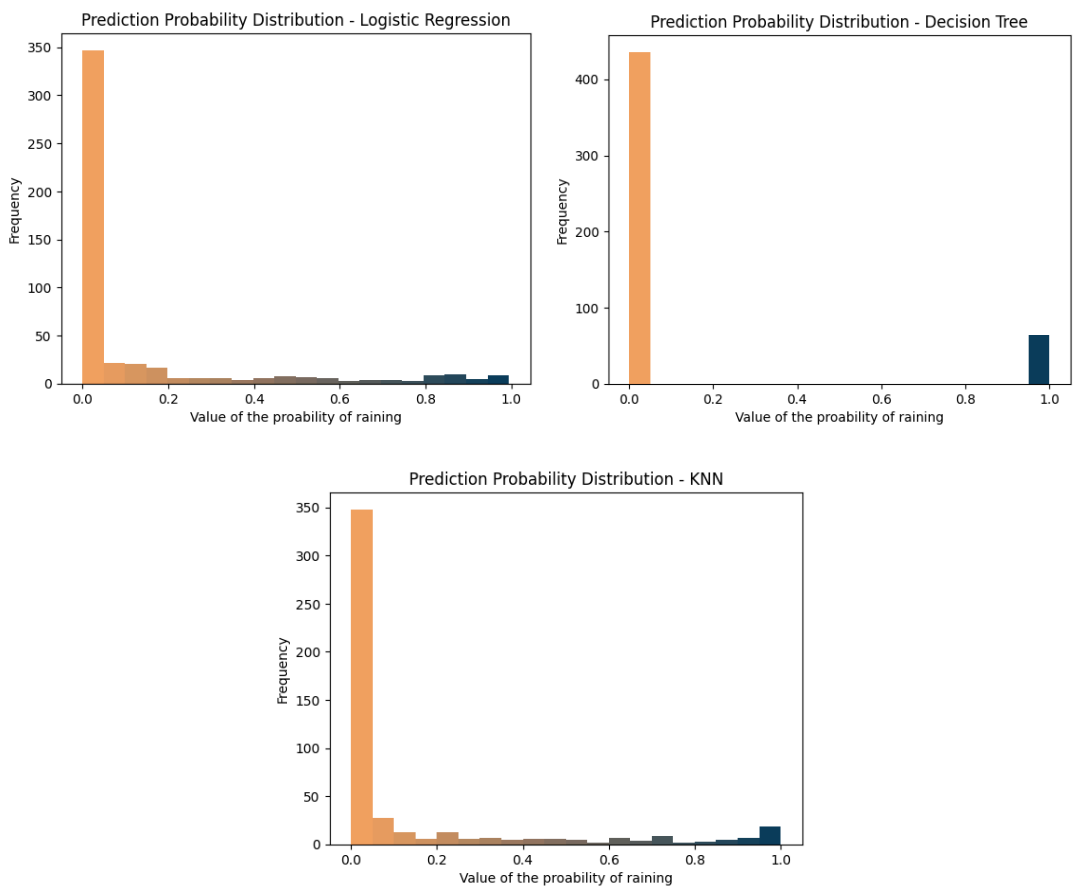

Figure 9: Prediction Probability Distributions across all 3 models.

Figure 9 provides a deeper look into the “certainty” of these predictions and highlights potential risks. The Decision Tree shows a polarized distribution, meaning all values are pushed to 0 or 1. This confirms its high accuracy but raised the concern of overfitting, which was discarded by confirming that the Tree model only used a depth of 3, which is shallow, suggesting that the model did indeed capture generalizable rules rather than memorizing noise. On the other hand, Logistic Regression displays significant uncertainty, with a considerable amount of frequent values between 0.2 and 0.8, suggesting that many weather instances are close to its linear decision boundary, making this model less reliable when making firm predictions. Lastly, KNN offers a middle ground, with its distribution showing high confidence but with a small margin of uncertainty.

6. Comparison with Related Work (Focus on Bank Telemarketing) :

The research paper “A Data-Driven Approach to Predict the Success of Bank Telemarketing” is written by Sérgio Moro, Paulo Cortez, and Paulo Rita. This research shares a description on predicting the success of bank telemarketing using machine learning techniques. In this section, we compare our results with their work using the same bank telemarketing dataset.

In our project we used three different classification models. Those are Logistic Regression, Decision Tree, and KNN. The model has good accuracy and their deference is not very far from each other. The Decision Tree model has earned the highest accuracy of 0.915, then the second highest accuracy is Logistic Regression with 0.911, and KNN being little behind compared to that model with accuracy of 0.897. We also used precision, recall, and F1 score to analyze the model, especially focusing on class 1 because the dataset is imbalanced.

In the research paper the authors used more complex models such as Neural Networks (NN), Support Vector Machines (SVM) in addition to Logistic Regression as well as Decision Tree. Their best model was the Neural Network; it achieved an AUC of around 0.929 during the modeling phase and about 0.794 during the final evaluation using rolling window validation. In our project, the models achieved higher AUC values around 0.93 for Logistic Regression and 0.92 for Decision Tree compared to the research paper. Even though our AUC value is higher

than the author does not mean we have the best model. This differs because the author used a rolling window method which is more realistic where our data split technique difficulty level is easy.

One of the major differences we have noticed in our work with the research paper is the evaluation method. In our project we used a simple train test split, 80% of data from the data set used for trains and the remaining 20% for testing. While the research paper used a more realistic rolling window evaluation. Their approach is very unique, which is to train models on past data but test on future data. This makes their results more realistic but also more difficult, which is why their AUC values lower to around 0.794, compared to ours because model cannot perfectly predict new patterns and mistake may happen,

The research paper was highly focused on AUC and LIFT metrics, which are more suitable for business decision-making problems like marketing. On the other hand we relied on accuracy, precision, recall, and F1-score. In the LIFT metric authors showed that their model could identify about 79% of the customers who would subscribe by selecting only 50% of the customers, which demonstrates strong performance in targeting the right audience. LIFT is more useful than accuracy in marketing problems because its main goal is to focus on the best potential customers rather than simply classify all cases. So, the direct comparison between our metrics and theirs is not always exact.

Another key difference is feature selection. The research paper started with 150 features and reduced them to 22 important features, which helped improve model performance and reduce complexity . In our project, we used all features after preprocessing and did not apply advanced feature selection. In terms of hyperparameter tuning, both works applied tuning techniques. In our project, we used GridSearchCV() and found that Decision Tree benefited the most from tuning. In the research paper, they also used grid search and heuristics to tune models such as Neural Networks and SVM.

Overall, both studies show similar patterns. Decision Tree performs well and Logistic Regression provides stable baseline performance. However, the research paper shows that more complex models such as Neural Networks can achieve better performance when using more advanced evaluation methods.

7. Conclusions

This project demonstrated a complete supervised machine learning workflow using the scikit-learn library across three distinct real-world classification problems: phishing website detection, bank telemarketing prediction, and weather forecast prediction. These datasets were selected to represent different domains such as cybersecurity, business analytics, and environmental prediction.

Three models were evaluated across all datasets: Logistic Regression, Decision Tree, and K-Nearest Neighbors (KNN). Overall, the Decision Tree model consistently performed best, particularly in capturing non-linear relationships and handling class imbalance. It achieved the highest accuracy across all datasets, including near-perfect performance in the weather dataset and strong results in both phishing and telemarketing tasks. Logistic Regression served as a stable baseline but struggled with complex patterns and minority class detection, while KNN performed well on cleaner datasets but was less effective on imbalanced data.

Additionally, our dataset selections allow complete test model performance under varying conditions such as class imbalance, mixed feature types, and dataset complexity. The results showed that dataset characteristics had a major impact on model performance. The phishing and weather datasets were relatively clean and structured, leading to high performance across all models, while the bank telemarketing dataset was more challenging due to class imbalance and mixed feature types, which reduced recall for the minority class. This highlighted the importance of using evaluation metrics beyond accuracy, such as precision, recall, F1-score, and ROC-AUC, especially for imbalanced datasets.

Hyperparameter tuning produced mixed results. It improved performance in some cases, particularly for KNN and the Decision Tree in certain datasets, but had little impact when models were already well-optimized. This indicates that tuning is beneficial but not always necessary depending on the model and data.

In comparison with existing research on the bank telemarketing dataset, our models achieved competitive performance, though differences in evaluation methods and model complexity make direct comparison limited. Overall, the Decision Tree model emerged as the most reliable across

all datasets, while Logistic Regression provided a consistent baseline and KNN showed strong but less stable performance.

A key takeaway for us from this work is that model performance is highly dependent on the nature of the dataset. Clean and well-structured datasets will tend to produce strong results across all models, while imbalanced or more complex datasets require additional care in preprocessing, model selection, and tuning. Moreover, this project highlights the importance of using multiple evaluation metrics rather than relying solely on accuracy, especially for imbalanced datasets. Metrics such as precision, recall, F1-score, and ROC-AUC provide a more complete understanding of model behavior.

Overall, this project demonstrated a complete supervised machine learning workflow, from data preparation and preprocessing through model training, hyperparameter tuning, and final evaluation. Across all datasets, each stage of the pipeline played an essential role in producing reliable results and meaningful comparisons between models.

8. Code

All 3 Jupyter Notebooks are available in our publicly accessible GitHub repository: https://github.com/lauramas6/project_164

9. References

Ahmad, Z. (2025, April). Weather Forecast Dataset, Version 1. Retrieved April 14, 2026 from https://www.kaggle.com/datasets/zeeshier/weather-forecast-dataset.

Chand, E. (2020). Phishing Website Detector Dataset (Version 3) [Dataset]. Retrieved April 10, 2026 from https://www.kaggle.com/datasets/eswarchandt/phishing-website-detector

Moro, S., Rita, P., & Cortez, P. (2014). Bank Marketing [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/C5K306.

10. Team Contributions

- **Manuel Domingo Mora**

  Phishing Website Detector Dataset

  Focus: *Classification of phishing vs legitimate websites*

  Contribution: Data preprocessing, model development, result analysis, report writing, PowerPoint and presentation.

- **Kazi Islam**

  Bank Telemarketing Dataset

  Focus: *Prediction of customer subscription to term deposits*

  Contribution: Data preprocessing, handling class imbalance, model development, hyperparameter tuning, result analysis, research analysis, and report writing.

- **Laura Mas**

  Weather Forecast Dataset

  Focus: *Prediction of rainfall based on weather conditions*

  Contribution: Data analysis, handling class imbalance, model development, result analysis, research, report writing, and GitHub repository handling.